Spec-Driven Development

Как договориться о поведении системы и проверить результат работы агента

Лекция 5 · AI Engineering · 2026

90 минут + 15 минут резерва



Avito Start

Маршрут

План: шесть крупных разделов

Понимаем метод → проходим workflow → смотрим инструмент → обсуждаем команду и доказательства.

Сейчас разберём

01 Теория → 02 Полный цикл OpenSpec

03 Демо Kiro → 04 SDD в команде

05 Данные и критика → 06 Практика

Затем: Крупные перебивки будут явно отмечать смену раздела.

01

Теория: как работает SDD

Сейчас разберём

Контракт, язык и разумный объём процесса.

Дальше: Полный цикл OpenSpec

Задаём агенту поведение обычным языком

В SDD мы фиксируем ожидаемое поведение в читаемой spec и управляем реализацией по этому договору.

01 Поручение «Напиши мне тесты» можно дать агенту обычным языком.

02 Spec сохраняет договор: что должно работать, где границы и как проверить результат.

03 Агент меняет код; человек принимает решения и проверяет результат.

Код, архитектура и тесты по-прежнему требуют инженерной проверки.

Смысл запроса нужно согласовать

Модель интерпретирует естественный язык. Это не детерминированная компиляция программы.

01 Неявные условия агент может заполнить своим предположением.

02 «Быстро», «корректно», «большой diff» требуют уточнения.

03 Пример и контрпример помогают увидеть разные трактовки.

04 SHALL и проверка формата не доказывают полноту или непротиворечивость смысла.

«Обработай несколько diff» — ещё не договор

Одиночный AI-review уже работает. Просим обработать сразу три изменения кода.

Поручение агенту · учебная ситуация

Сохраняем работу одиночного review.

Поведение при частичной ошибке ещё не выбрано.

При одной ошибке возможны разные результаты

Одному поручению подходят разные реализации. Нужное поведение выбирает человек.

Остановиться

Вернуть ошибку b и готовый результат a.

c не вызывать.

Продолжить

Получить результат каждого элемента.

b пометить ошибкой.

Запустить параллельно

Потребуется договориться об отмене и порядке.

В наше изменение не входит.

Фиксируем выбранное поведение в spec

Spec — сохранённое описание ожидаемого поведения, границ и примеров проверки.

01 Пакет: 1–3 элемента с уникальными строковыми ID.

02 Проверяем весь вход до первого reviewer call.

03 Выполняем последовательно; сохраняем ID и порядок.

04 При первой ошибке останавливаемся; одиночный endpoint сохраняет поведение.

Примеры делают договор проверяемым

Сценарий задаёт ожидаемый результат. Его нужно превратить в проверку поведения.

Ситуация	Ожидаемое поведение
3 валидных item	3 результата в исходном порядке
Повторный ID	422; reviewer calls = 0
Последний diff пуст	422; reviewer calls = 0
Ошибка на b в [a,b,c]	a сохранён; b назван; c не вызван
Одиночный review	Прежний контракт сохраняется

Текст примера ещё не означает, что проверка выполнена.

Spec читают и агент, и человек

Для агента это рабочий вход. Для человека — договор, который можно рассмотреть и принять.

01 Короткое описание цели и границ.

02 Проверяемые требования и значимые примеры.

03 Технические решения и причины — отдельным уровнем.

Поручение, спецификация и процесс SDD

Это три разные роли, а не последовательность типов документа.

Prompt

Конкретное поручение агенту.

Может ссылаться на код и согласованную spec.

Spec

Сохранённый договор о поведении, границах и примерах.

Его читают исполнитель, reviewer и человек.

Процесс SDD

Согласовать → реализовать → проверить → обновить договор.

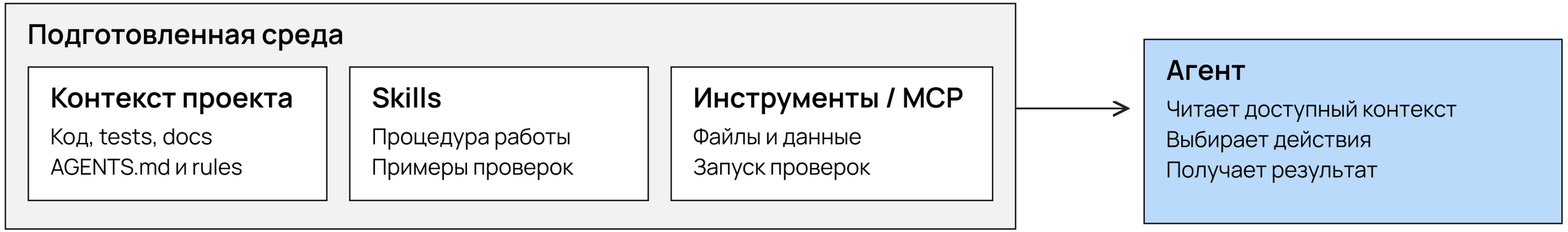
Новые факты возвращают к обсуждению.

Согласованная spec становится частью контекста

Добавляем договор конкретного изменения к существующим коду, правилам, инструментам и проверкам.

«Напиши мне тесты»

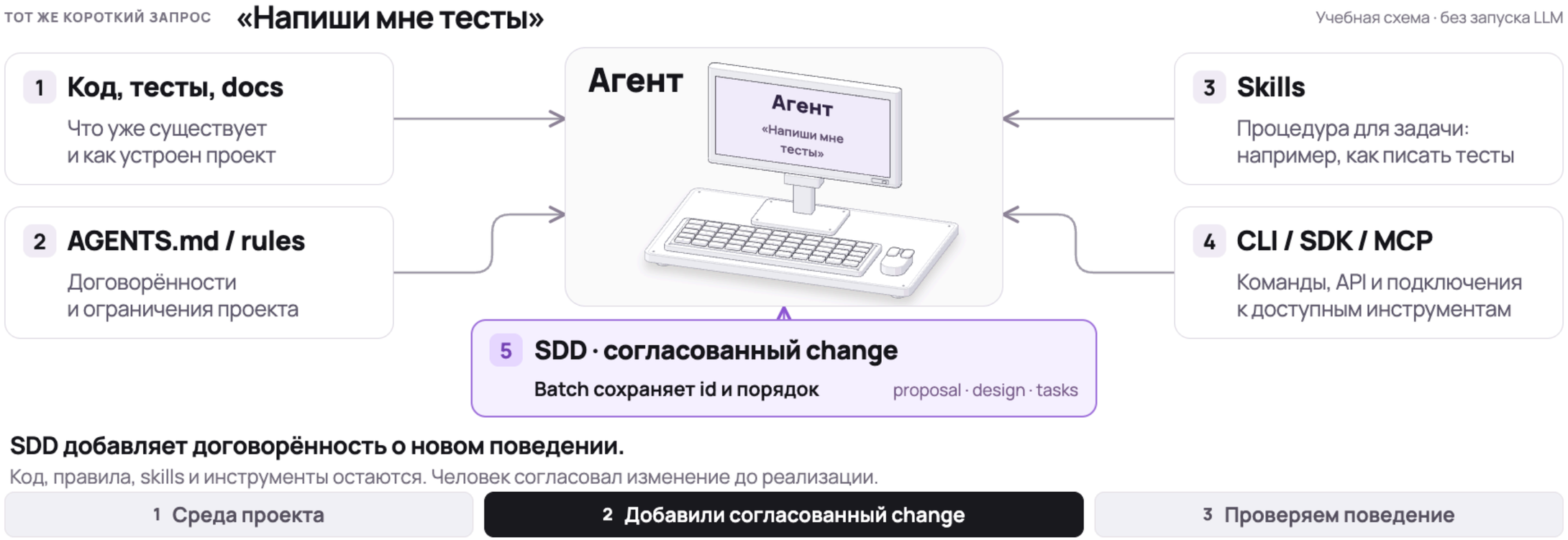
Поручение остаётся коротким



Правила, процедура и инструменты уже доступны агенту. Короткий запрос опирается на эту среду.

Среда связывает намерение, инструменты и проверку

Spec задаёт ориентир для агента. Код, tools и checks позволяют реализовать и проверить договор.



Модель рабочего окружения агента · prepared illustration, не live LLM

SDD возможен без отдельного framework

Plan mode или свой spec.md подходят, если договор ведёт реализацию и приёмку.

План действий

Создать handler.

Добавить тесты.

Обновить README.

Договор поведения

Неверный item → ни одного вызова.

Ошибка b → c не запускается.

Старый endpoint совместим.

Свой процесс или framework

Plan mode и свой spec.md могут выполнять роль SDD.

Договор должен вести реализацию и приёмку; framework упорядочивает этот цикл.

Сколько процесса нужно изменению

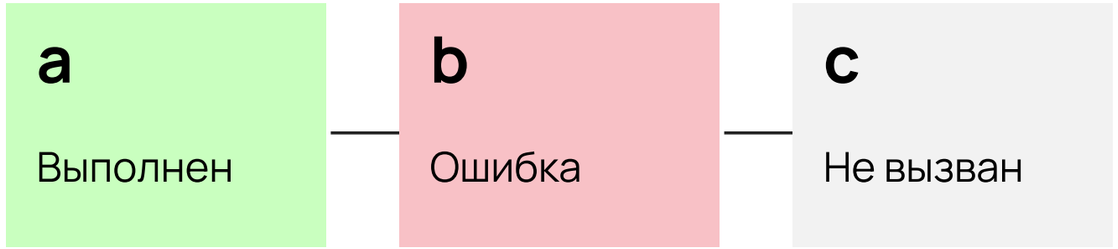
> **Обсуждение до кода стоит времени. Оно может снизить переделки, но не гарантирует экономию.**

Изменение	Достаточная опора
Опечатка / очевидная правка	Короткий запрос + проверка
Локальный bug	Симптом + regression test + review
Новая ветка поведения	Контракт + примеры + реализация + приёмка
Несколько систем / сложный риск	Контракт + архитектура + поэтапная проверка

Fail-fast: что увидит клиент

POST /api/review-batches: при ошибке b возвращаем готовый a, а c не запускаем.

1 · ошибка второго item · HTTP 502



Результат a остаётся в ответе.

После ошибки b работа останавливается.

Учебный пример контракта. Реальный прогон проверяем отдельно.

Simplified Technical English

ASD-STE100 упорядочивает английский технический текст, когда формулировка оставляет разные трактовки.

Неясно

Handle failures gracefully and return useful results.

Явнее

If the reviewer fails, stop the batch.

Return the failed item ID and completed results.

Community skills адаптируют STE для agent-facing English; это не официальная сертификация.

EARS: условие → реакция

Когда теряется условие, явно свяжите событие и реакцию системы.

Учебный пример EARS, не цитата

Gherkin, BDD и TDD

Разные задачи: записать пример, согласовать поведение и развивать код через проверку.

Средство	Для чего	Наш пример
Gherkin	Записать сценарий	Given a,b,c; When b fails; Then c is not called
BDD	Согласовать поведение	Обсудить fail-fast с примерами
TDD	Развивать код через проверку	Failing test → code → refactor

Диаграмма выбирается под вопрос

Когда текст скрывает связи или порядок, выберите подходящую диаграмму.

01 C4: где находится сервис и с чем он связан.

02 Sequence diagram: кто кого вызывает и когда.

03 Mermaid: способ хранить диаграмму текстом рядом с кодом.

Текст сценария → диаграмма

Порядок вызовов можно хранить в читаемом текстовом источнике.

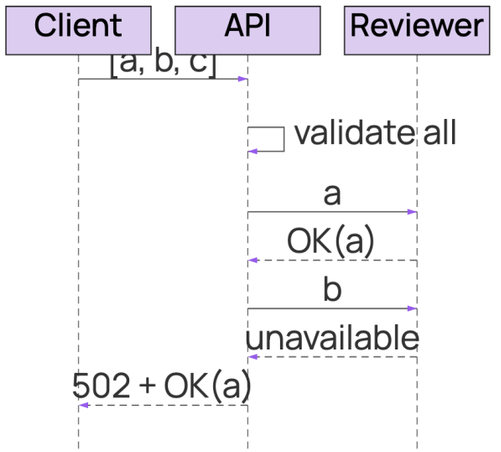
Один сценарий: исходник и его схема

Только схема ↗

Редактируемый contract.mmd

```
sequenceDiagram
participant C as Client
participant A as API
C->>A: [a, b, c]
A->>A: validate all
A->>Reviewer: a
Reviewer-->>A: OK(a)
A->>Reviewer: b
Reviewer-->>A: unavailable
A-->>C: 502 + OK(a)
```

SVG, построенный из этого исходника



Fail-fast: весь вход валиден; reviewer недоступен на b. Результат a сохранён, c не вызывается.

Подготовленный сценарий контракта · визуализация: beautiful-mermaid skill + beautiful-mermaid 1.1.3 · это не trace реального запроса

Исходный текст и его диаграмма · учебный контракт batch API

Сначала познакомиться с системой

Спецификация должна учитывать то, что уже существует.

01 Найти путь запроса и границы компонентов.

02 Прочитать действующие tests и API-контракт.

03 Проверить baseline, команды и ограничения.

04 Отделить наблюдаемое поведение от желаемого.

Новый и существующий проект

В обоих случаях нужны контекст, архитектура и критерии приёмки.

Greenfield

Определить продукт и границы.

Выбрать минимальную архитектуру.

Создать первые проверяемые сценарии.

Brownfield

Проверить действующее поведение.

Найти существующие решения.

Зафиксировать сохранение совместимости.

Вывод: договор направляет агента

В SDD ожидаемое поведение становится явным договором с агентом.

Польза	Условия и примеры дают человеку и агенту общее ожидание результата.
Ограничение	Естественный язык неоднозначен; наличие текста не означает выполненную проверку.
Что делаем	Начинайте с ясного ожидания и минимального процесса, который можете проверить.

02

OpenSpec: полный цикл

Сейчас разберём

Пройдём изменение: от идеи до принятого поведения и актуальной spec.

Дальше: Выбор инструмента и демо

OpenSpec: три состояния знания

Текущий договор, предложенное изменение и история.

openspec/specs/

Текущее согласованное поведение.

openspec/changes/

Proposal, design, tasks и delta specs активного изменения.

changes/archive/

История завершённых изменений и их решений.

Цикл изменения и решения человека

Команды помогают двигаться. Смысл и приёмку определяет человек.

Планируем изменение

Получаем и проверяем результат



Сначала делаем намерение и границы изменения видимыми. Артефакты можно уточнять по мере понимания задачи.

Explore: выяснить неизвестное

До предложения выясняем, какие решения действительно нужны.

В чате агента · canonical OpenSpec syntax

Propose: черновик для обсуждения

Агент оформляет изменение. Мы проверяем принятые в нём решения.

01 Proposal: зачем и что входит в изменение.

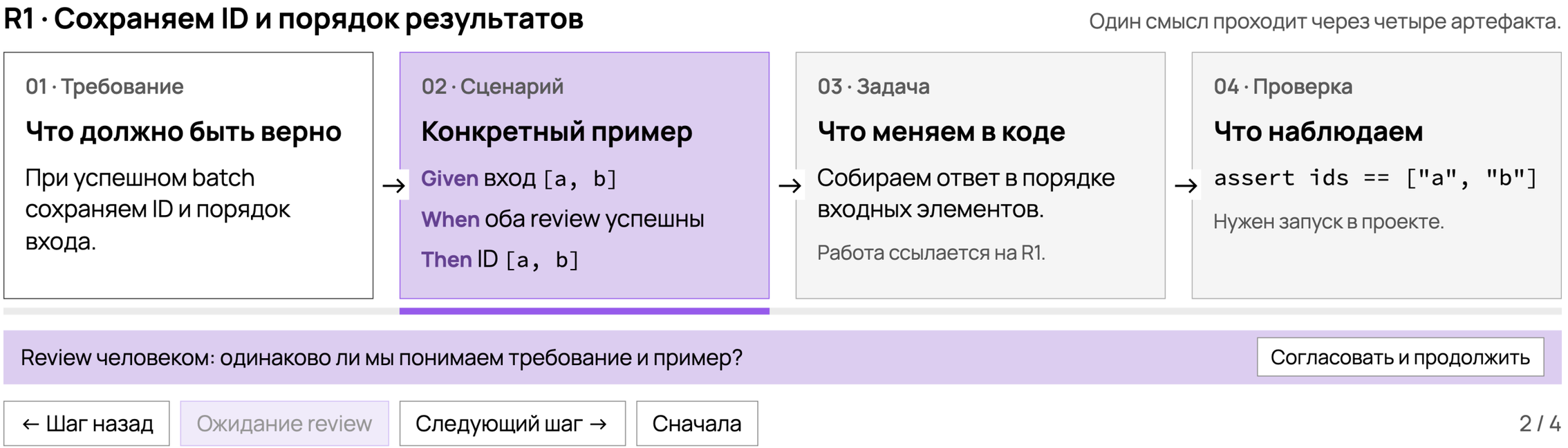
02 Specs: требуемое поведение и сценарии.

03 Design: технический подход и причины.

04 Tasks: работа, которую нужно выполнить.

Как требование доходит до проверки

Связь полезна, когда по ней можно найти и проверить конкретный результат.



Подготовленная учебная связь, не автоматическая проверка покрытия · подготовка: официальный GSAP timeline skill · воспроизведение: локальный GSAP 3.15.0

Подготовленные связи одного требования · без автоматической гарантии покрытия

Review до кода

Проверяем решения, а не красоту Markdown.

01 Что наблюдает пользователь при успехе и ошибке?

02 Какой важный случай текст оставил на усмотрение агента?

03 Не поменялось ли старое поведение без причины?

04 Можно ли по этим условиям принять реализацию?

Один пробел → одно явное решение

Неверный последний item не должен запускать первые review.

До review

Сервис проверяет каждый item и возвращает ошибку для некорректного ввода.

После review

Сервис валидирует весь пакет до первого вызова reviewer.

Если b пуст в [a,b], HTTP 422 и reviewer calls = 0.

Обсуждение стоит времени; оно может снизить переделки, но не гарантирует экономию.

Apply: реализовать принятый change

Агент использует договорённости и выполняет задачи.

В чате агента

Во время реализации появились новые факты

Возвращаемся к решению, если старое предположение больше не работает.

01. →

Факт

Нашли ограничение в коде

02. →

Решение

Уточнили поведение с человеком

03. →

Артефакты

Обновили связанные договорённости

04.

Код и тесты

Реализовали и перепроверили

Исполнитель и независимый reviewer

Два агентских прохода помогают разделить создание и проверку.

Исполнитель

Меняет код и тесты.

Показывает diff и фактические проверки.

Reviewer

Читает контракт и изменения.

Ищет дефекты, пробелы и регрессии.

Человек

Разбирает findings.

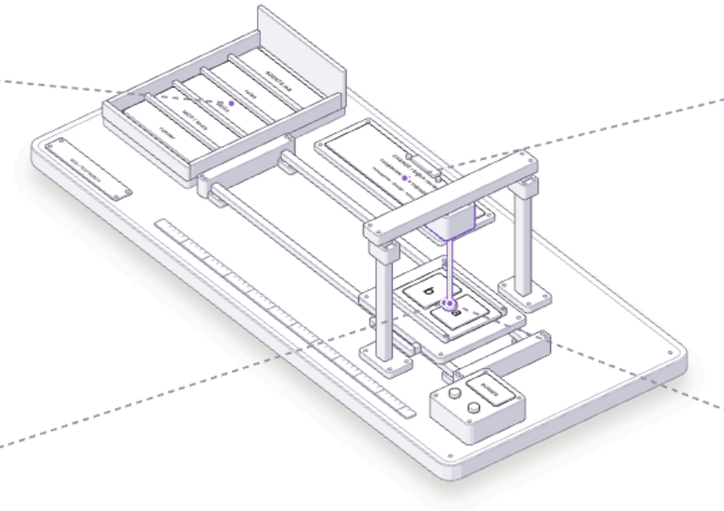
Принимает смысл и итоговый результат.

Тест сравнивает ожидание с результатом

Ожидали a, b. Получили b, a. Проверка должна показать FAIL.

Проверяем порядок: a, b

Контекст проекта
AGENTS.md · rules
skills · MCP/tools



Требование
Сохранить id
и порядок a, b

Проверяемый код
Результат на каретке

Runner
Выполняет тест
и сравнивает порядок

Тест сравнивает требование с поведением кода.

Учебная модель: агент предлагает тест, runner проверяет заданный сценарий. Это подготовленная симуляция, без обращения к LLM.

Тот же короткий запрос
«Напиши мне тесты»

Среда | Review | Тест

Тест обнаружил ошибку

Согласование требования не исправило дефект: reverse() меняет порядок.

Batch сохраняет id и порядок
✓ Человек согласовал требование

Ожидаем `['a', 'b']`
Получили `['b', 'a']`

FAIL · порядок нарушен

ТЕСТ, ПРЕДЛОЖЕННЫЙ АГЕНТОМ

```
assert actual_ids == ['a', 'b']
```

Повторить тест | Вручную: исправить код

Начать заново — Учебная модель, без LLM

Учебная симуляция проверки порядка · expected a,b → actual b,a → FAIL

Приёмка опирается на поведение

> **Галочка в tasks.md не равна выполненному требованию.**

01 Запущены проверки нужной версии кода.

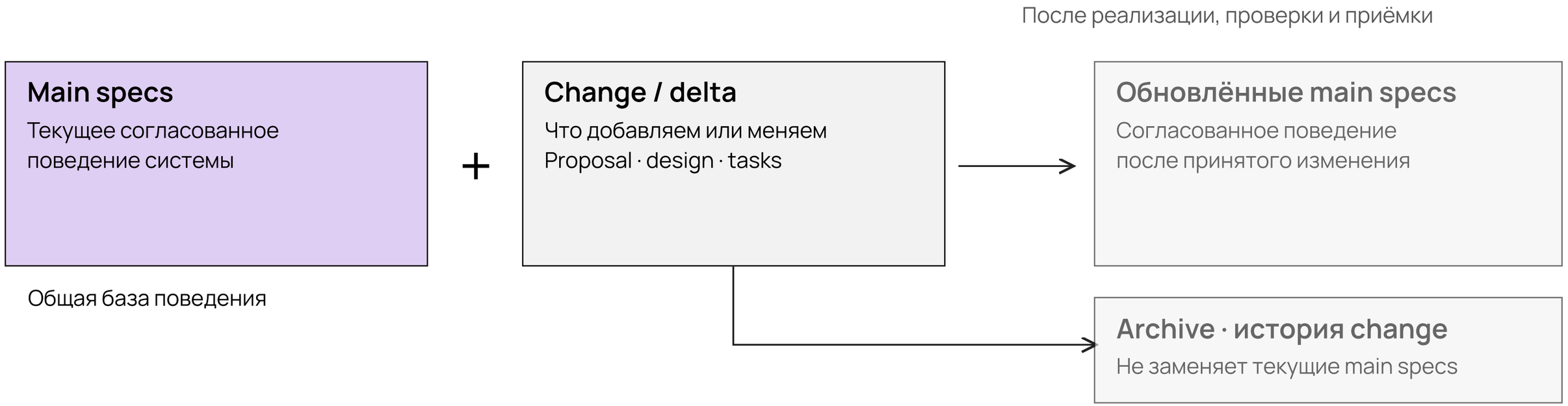
02 Тест различает корректное и ошибочное поведение.

03 Старый endpoint прошёл regression checks.

04 Неизвестное и непроверенное названы явно.

Sync и archive

Согласованное изменение становится частью текущей спецификации.

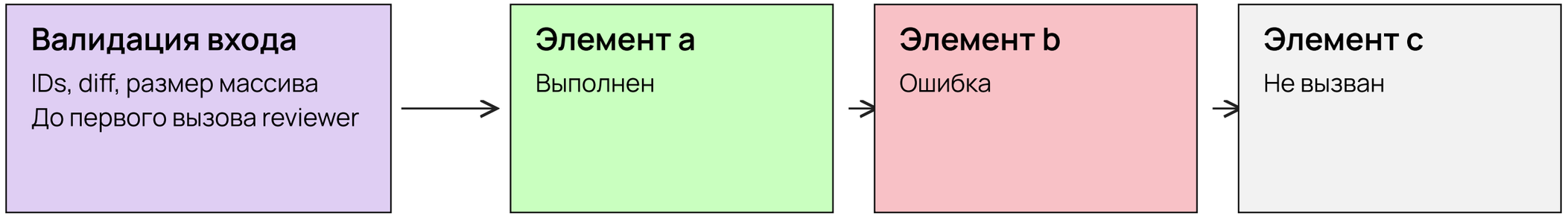


Main specs описывают текущее поведение. Новое намерение ещё не становится фактом системы.

Следующий change: продолжать после ошибки

Новая опция сохраняет старое поведение по умолчанию.

Один запрос · три элемента · ошибка reviewer на b



Режим fail-fast · HTTP 502

results: [a] · error: reviewer_unavailable, id: b

Fail-fast: a выполнен, b завершился ошибкой; c не вызывается. Ответ 502 сохраняет уже полученный результат a.

Настроить OpenSpec под свой проект

Добавляем сведения, которые меняют качество следующего предложения.

openspec/config.yaml · учебный фрагмент

Реальный стек и команды проверок.

Ссылка на действующий API и архитектуру.

Требование сохранять совместимость.

Правило описывать значимые ошибки и границы.

Правила, workflow и реальные ограничения

Текстовая инструкция и исполняемая проверка действуют по-разному.

Механизм	Назначение
context / rules	Контекст и инструкции для артефактов
schema	Состав документов и зависимости
operations.apply.guidance	Рекомендации по выполнению apply
Tests / CI / permissions	Исполняемые проверки и ограничения

Меняйте форму осознанно: сохраняйте структуру, которую ожидает парсер, либо адаптируйте schema.

Знания о продукте и причины решений

Код показывает реализацию. Для следующего решения нужен ещё контекст.

01 Что делает продукт и для кого.

02 Какие ограничения нельзя нарушить.

03 Почему выбран fail-fast и что оставлено за scope.

04 Где находится актуальная проверка.

Drift: текст, код и тесты разошлись

> **Долг появляется, когда следующая работа опирается на неверное описание.**

01. →

Событие

Изменили API, код или решение

02. →

Проверка

Нашли затронутые specs и tests

03. →

Исправление

Обновили в том же change

04.

Уборка

Периодически удаляем накопленные дубли

Четыре способа организовать SDD

Выбираем подход под работу команды и привычную среду.

Инструмент	Разработчик	Сильная сторона	Что учитывать
OpenSpec 1.14.1	Fission AI	Delta, main specs, archive	Своя дисциплина приёмки
Spec Kit 1.1.3	GitHub	Constitution и этапы разработки	Глубина артефактов
Kiro	AWS	Specs и выполнение вместе	Среда и workflow
BMad 6.12.0	Brian Madison / community	Роли и workflows	Нужный объём процесса

Как выбрать framework

Попробуйте один небольшой change и сравните полную стоимость.

01 Насколько легко читать и исправлять draft?

02 Что происходит после нового факта или второго изменения?

03 Как workflow использует существующие tests и conventions?

04 Сколько времени уходит на review, исправления и поддержку?

Вывод: документы не заменяют приёмку

Framework организует цикл; человек принимает результат по проверкам поведения.

Польза	Сохранять связь решения, реализации, проверок и актуальной spec.
Ограничение	Документы, tasks [x] и успешный archive не доказывают нужное поведение.
Что делаем	При новых фактах возвращайтесь к решению; после приёмки обновите договор.

03

Демо: Kiro

Сейчас разберём

Посмотрим, как обсуждать требования и переходить к design и tasks.

Дальше: SDD в команде

Kiro: specs внутри рабочего инструмента

Requirements, design и tasks доступны как части одного workflow.

01 Требования могут использовать EARS.

02 Design содержит архитектуру и взаимодействия.

03 Tasks связывают план с выполнением.

04 Specs доступны в IDE, CLI и Web.

Kiro: живой показ одного решения

Сначала читаем требования, затем смотрим переход к design и tasks.

1. Открыть подготовленный baseline.
2. Попросить batch review и прочитать draft.
3. Уточнить полную валидацию до reviewer calls.
4. Проследить, как решение попало в design и tasks.

4 мин

Результат этапа

Сохраните результат и проверьте критерии готовности.

04

SDD в команде: процесс и внедрение

Сейчас разберём

Распределим решения, review, проверки и обновление знаний.

Дальше: Данные и критика

Как начать SDD в команде

Рекомендуемый сценарий: один пилотный change и понятная ответственность.

01.

Пилот

Одна фича, исходное поведение и критерии

02.

Обсуждение spec

PM / аналитик / QA уточняют примеры с разработчиком

03.

Реализация

Агент и разработчик; review кода

04.

Приёмка

Проверки CI и решение человека

05.

Знания

Актуальные specs, причины решений и владелец обновления

Audible: сначала review спецификации

Пример процесса одной команды, рассказанный её участником.

01. →

Spec PR

Review до кода

02. →

Apply

Реализация

03. →

Code review

Проверка результата

04.

Archive

Человек завершает change

В рассказе — Mermaid и TDD; использование Gherkin не подтверждено.

Границы рассказа Audible

Опыт команды не отвечает за все команды и версии инструментов.

01 PM пока не работает с OpenSpec напрямую.

02 Lead использует PRD и вход от PM.

03 Число итераций code review, по рассказу, не изменилось.

04 Долгосрочная работа с архивом ещё не подтверждена.

Вывод: знания требуют владельца

> **Командная ценность spec — согласованные решения и причины, доступные следующему участнику.**

Польза	Обсуждать ожидания между PM, разработкой и QA; передавать контекст.
Ограничение	Чтение и обновление стоят времени; Audible — частный опыт, не общее обещание ускорения.
Что делаем	Начните с пилота, назначьте владельца и обновляйте знания вместе с изменением.

05

Данные и критика SDD

Сейчас разберём

Разделим измеренные результаты, частный опыт и ограничения подхода.

Дальше: Практика

Google: code → spec → tests

Контролируемый эксперимент по генерации тестов на 90 исторических bugs.

01. →

Исправленный код

Без прежних tests и fix diff

02. →

Два варианта

С контрактом или без него

03. →

Новые tests

Одинаковые model и harness

04.

Проверка bug

Запуск на fixed и buggy code

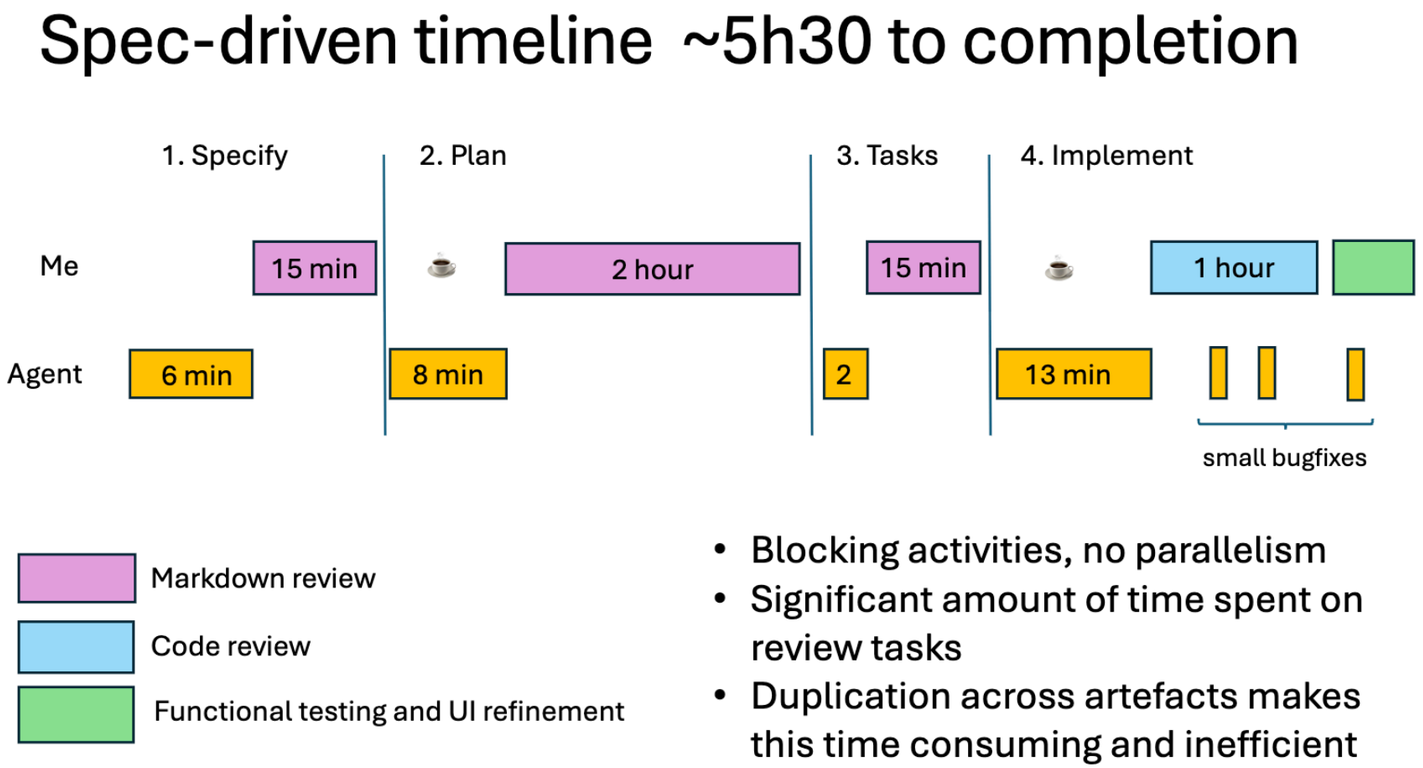
Результат исследования и его границы

Контракт помог в этой задаче и увеличил расход tokens.

Метрика	Изменение
Обнаружение исторических bugs · detect@5	+9,8 п.п.
Branch coverage	+2,5 п.п.
Суммарные tokens	+38%
Что не измеряли	Продуктивность команды / внедрение framework

Scott Logic: время человека тоже входит в цену

Оригинальная схема автора, ноябрь 2025. Один проект и конкретный процесс.



Colin Eberhardt · Scott Logic · 26.11.2025 · исходная схема без изменений

Что видно

Review Markdown занимает время человека, пока агент ждёт следующего шага.

Что это означает

Стоимость процесса включает чтение, исправления и проверку результата.

Граница вывода

Один кейс 2025 года. Неравные scope нельзя превращать в универсальное «SDD в 4 раза медленнее».

Критика: документы можно перестать читать

Если объём растёт быстрее способности проверять, SDD теряет контроль.

01 Текст повторяет код и быстро устаревает.

02 Формальная структура скрывает спорные решения.

03 Review превращается в одобрение непрочитанного.

04 Стоимость поддержания превышает пользу для изменения.

Разрыв между spec и кодом остаётся

Совпадение красивых описаний не доказывает корректность программы.

Spec

Требование может быть неполным или противоречивым.

Проверка формата не проверяет смысл.

Code

Агент может неверно реализовать даже понятное требование.

Проверка

Тест может повторить ошибку реализации или не затронуть нужный случай.

Хороший код и архитектура остаются основой

Качество кода, архитектуры и проверок остаётся основой работы агента.

01 Простая архитектура и ясные границы модулей.

02 Строгие типы и чистые функции для важной логики.

03 Внешние эффекты изолированы; tests воспроизводимы.

04 Линтеры, conventions и рабочий harness дают обратную связь.

Другой путь: извлекать знания из кода

Навигация, AST, tests и поведение тоже дают агенту опору.

01. →

Код / AST

Структура и связи

02. →

Tests / trace

Наблюдаемое поведение

03. →

Контракт задачи

Уточнённое ожидание

04.

Решения / why

Контекст, которого нет в коде

Часть контракта закрепляем в коде

Пример batch API: успех требует report, ошибка — отдельный error.

Учебный пример · TypeScript

DDD: выяснить, что успех и ошибка означают в предметной области.

DSL / схема: выразить машинопроверяемую часть правил.

ADT и строгие типы: явно различить допустимые результаты.

Scaffolding и сила модели

Польза структуры зависит от модели, задачи и среды.

01 Структура может помочь не пропустить важные вопросы.

02 Она может добавить лишнюю работу и tokens.

03 Более сильная модель не отменяет договор о продукте.

04 Более слабая модель не становится надёжной от шаблона.

Вывод: сохраняем смысл и проверку

> **Frameworks могут меняться; ясность намерения, проверяемый результат и причины решений остаются опорой.**

Польза	Связать намерение с кодом/tests и сохранить контекст для следующего участника.
Ограничение	Spec не заменяет архитектуру и проверки; лишний текст может увеличить стоимость.
Что делаем	Сравнивайте полное время до принятого результата: генерацию, review и исправления.